

1. Audit de l'existant

1.1 Objectif et critères d'analyse

Cet audit a pour objectif d'analyser les applications existantes de Your Car Your Way afin d'identifier leurs forces, leurs faiblesses et leurs contraintes techniques.

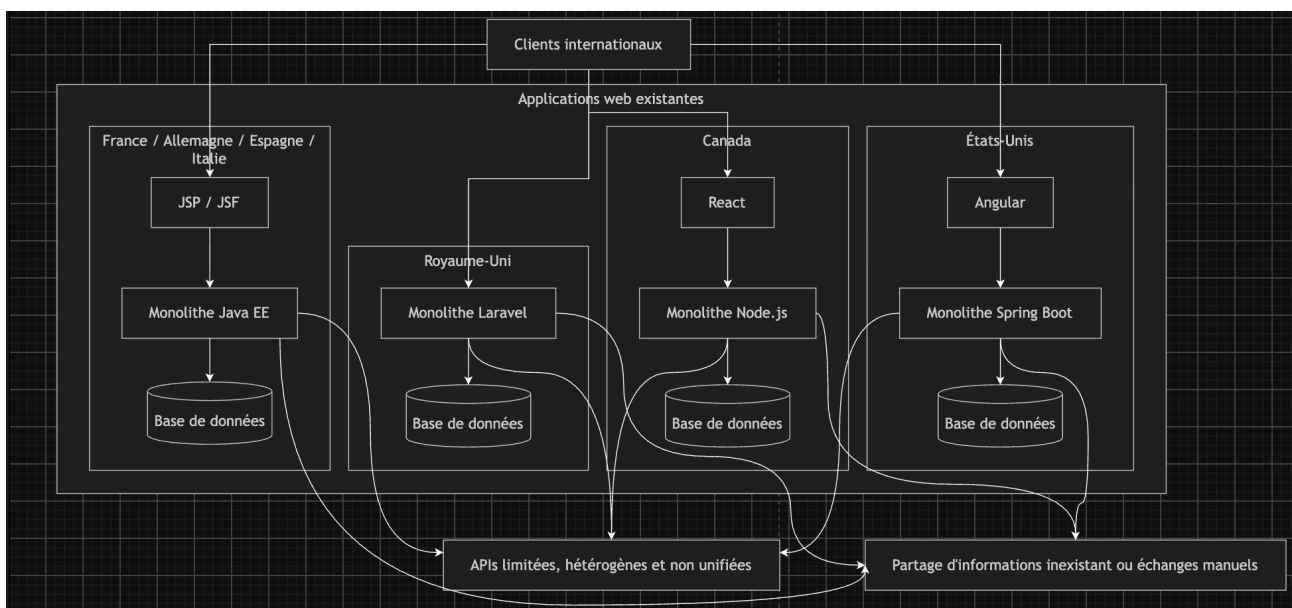
L'analyse repose sur les critères suivants : maintenabilité, performance, évolutivité, fiabilité/disponibilité, sécurité et cohérence technique.

1.2 Architecture actuelle

Your Car Your Way dispose actuellement de quatre applications distinctes, développées indépendamment selon les pays. Elles reposent principalement sur des architectures monolithiques et utilisent des stacks technologiques différentes : Java EE/JSP/JSF pour les applications historiques France, Allemagne, Espagne et Italie, Laravel/PHP au Royaume-Uni, React/Node.js au Canada et Angular/Spring Boot aux États-Unis.

Chaque application possède son propre modèle de données. Les APIs sont limitées et non unifiées, tandis que les échanges d'informations entre applications sont inexistantes ou réalisés manuellement.

Diagramme de composants haut niveau — AS-IS



1.3 Tableau d'analyse — Critères x architectures

Tu peux utiliser un tableau plus compact :

Critère	FRA / ALL / ESP / ITA	ENG	Canada	USA
Stack	Java EE, JSP/JSF	Laravel/PHP	React/Node.js	Angular/Spring Boot
Maintenabilité	Faible : code et fonctionnalités divergents	Moyenne	Moyenne	Bonne
Performance	~150 req/s, jusqu'à 4 % d'erreurs	~250 req/s, 1,5 %	~300 req/s, 1,5 %	~350 req/s, 0,8 %
Disponibilité	97,2 %	98,6 %	98,1 %	98,9 %
Évolutivité	Limitée	Limitée	Limitée	Limitée
Sécurité	SHA-1, TLS 1.0, secrets en configuration	bcrypt, rotation limitée	argon2id, rotation limitée	bcrypt, Key Vault partiel
Déploiement	Manuel / OVH	AWS EC2	AWS	Azure / containers
Données / intégration	Schémas séparés, APIs limitées	Isolée	Isolée	Isolée

1.4 Forces, faiblesses et contraintes

Forces

- **Des architectures techniquement plus récentes existent déjà.** Le Canada et les États-Unis utilisent des stacks modernes, respectivement **React/Node.js** et **Angular/Spring Boot**, tandis que les applications historiques reposent encore sur Java EE/JSP/JSF.
- **Les performances** sont meilleures sur les applications les plus récentes. L'application américaine atteint environ 350 requêtes par seconde, contre environ 300 req/s au Canada, 250 req/s au Royaume-Uni et seulement 150 req/s pour les applications France/Allemagne/Espagne/Italie.
- **La disponibilité** est globalement meilleure sur les architectures récentes. L'application américaine atteint 98,9 % de disponibilité, contre 98,1 % au Canada, 98,6 % au Royaume-Uni et 97,2 % pour les applications historiques européennes.
- **Les environnements cloud offrent des infrastructures plus adaptées à l'évolution du système.** Le Royaume-Uni, le Canada et les États-Unis reposent sur AWS ou Azure, tandis que les applications historiques utilisent principalement un hébergement OVH avec des déploiements manuels.

- Certaines pratiques de **sécurité** sont déjà adaptées aux standards actuels. Le Canada utilise notamment Argon2id pour le stockage des mots de passe et les États-Unis utilisent bcrypt ainsi qu'Azure Key Vault pour une partie de la gestion des secrets.

Faiblesses

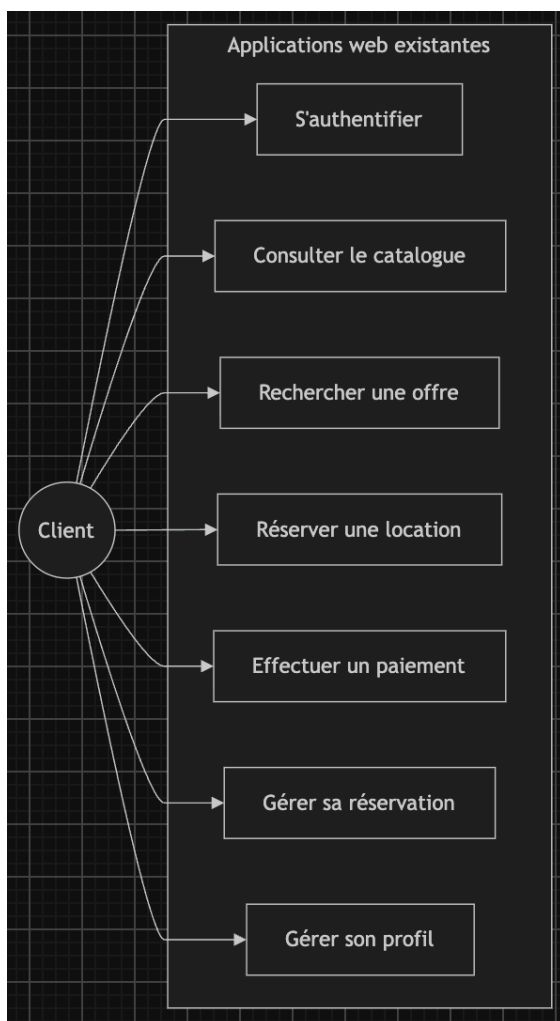
- **Forte hétérogénéité technologique.** Les applications reposent sur quatre stacks principales : Java EE/JSP/JSF, Laravel/PHP, React/Node.js et Angular/Spring Boot. Cette diversité augmente la complexité de maintenance et nécessite des compétences techniques différentes selon les applications.
- **Maintenabilité inégale.** Les applications France/Allemagne/Espagne/Italie présentent une maintenabilité faible en raison de la divergence progressive du code et des fonctionnalités. Le Royaume-Uni et le Canada présentent une situation intermédiaire, tandis que les États-Unis disposent de la situation la plus favorable.
- **Écarts importants de performance.** Les applications historiques atteignent environ 150 req/s, soit moins de la moitié des performances observées aux États-Unis (350 req/s). Elles peuvent également atteindre jusqu'à 4 % d'erreurs lors des pics, contre 0,8 % aux États-Unis.
- **Disponibilité hétérogène.** L'écart entre les applications historiques (97,2 %) et l'application américaine (98,9 %) montre des niveaux de disponibilité différents selon les environnements.
- **Évolutivité limitée.** Les quatre architectures reposent sur des applications monolithiques et aucune ne propose actuellement une architecture conçue pour mutualiser facilement les fonctionnalités et les données à l'échelle internationale.
- **Divergence des applications européennes.** Même si les applications France, Allemagne, Espagne et Italie sont issues d'une base historique commune, elles ont progressivement divergé au niveau du code, des fonctionnalités et des règles métier.
- **Divergence des logiques métier entre les pays.** La mutualisation technique ne peut donc pas se limiter à copier l'une des applications existantes : certaines spécificités métier doivent être conservées ou harmonisées dans la future solution.
- **Données cloisonnées.** Chaque application possède sa propre base de données et son propre schéma, ce qui complique le partage et la centralisation des informations.
- **Intégration limitée.** Les APIs existantes sont limitées et non unifiées, ce qui rend les échanges entre applications complexes.
- **Sécurité hétérogène.** Les applications historiques présentent les principales faiblesses, notamment l'utilisation de SHA-1, de TLS 1.0 et le stockage de certains secrets dans la configuration. Les mécanismes plus récents utilisés au Canada et aux États-Unis ne sont par ailleurs pas uniformisés à l'ensemble du système.

Contraintes techniques

- **Hétérogénéité des technologies et infrastructures.** La future architecture devra tenir compte de l'existant composé de Java EE, Laravel, Node.js, Spring Boot, ainsi que des environnements OVH, AWS et Azure.
- **Migration des données existantes.** Les données étant réparties dans plusieurs bases utilisant des schémas différents, leur consolidation nécessitera une stratégie de migration et de transformation des données.

- **Harmonisation des règles métier.** Les différentes applications présentent des logiques métier divergentes. La future solution devra donc distinguer les fonctionnalités communes des spécificités propres à certains marchés.
- **Continuité de service.** La migration vers une nouvelle architecture devra limiter les interruptions de service et permettre une transition progressive depuis les applications existantes.
- **Maintien de la compatibilité avec l'existant.** La future architecture devra prévoir des mécanismes d'intégration permettant de communiquer avec les systèmes existants pendant la période de transition.
- **Sécurisation de la migration.** Les données utilisateurs et notamment les données liées aux comptes et aux réservations devront être transférées de manière sécurisée.
- **Unification progressive.** La future architecture devra permettre de centraliser les fonctionnalités et les données sans supposer que toutes les règles métier actuelles peuvent être uniformisées immédiatement.

1.5 Cas d'utilisation UML



1.6 Conclusion de l'audit

L'audit met en évidence un système fonctionnel mais fortement fragmenté. Les applications existantes présentent des niveaux de maturité technique différents et reposent sur des technologies, infrastructures et modèles de données hétérogènes.

Cette fragmentation limite la maintenabilité et l'évolutivité globale du système. Les écarts constatés en matière de performance, de disponibilité et de sécurité renforcent également la nécessité d'harmoniser les futures pratiques techniques.

Parmi les architectures étudiées, le projet américain apparaît comme celui qui se rapproche le plus d'une base techniquement envisageable pour la future application. Il dispose notamment d'une stack plus moderne, plus sécurisée, plus performante et plus disponible. Toutefois, il ne peut pas être repris tel quel comme architecture cible.

En effet, même si les applications européennes présentent une certaine similitude historique et fonctionnelle, notamment du fait de leur évolution à partir d'une même application, des divergences de logiques métier existent également entre les différents pays. Cette situation montre que la simple uniformisation technique des applications existantes ne permettrait pas à elle seule de répondre aux besoins de l'ensemble des marchés.

La future architecture devra donc s'appuyer sur les enseignements de l'existant plutôt que reproduire l'une des architectures actuelles. Elle devra notamment répondre aux principaux enjeux identifiés : centralisation des usages, cohérence technique, prise en compte des spécificités métier internationales, standardisation des échanges et des données, amélioration de la sécurité et capacité d'évolution à l'échelle internationale.

2. Architecture cible et choix techniques

2.1 Spécifications techniques

La nouvelle application doit répondre aux besoins fonctionnels identifiés tout en permettant de centraliser les différents marchés de Your Car Your Way.

PERFORMANCE

L'application doit pouvoir supporter une montée en charge progressive avec l'augmentation du nombre d'utilisateurs et de réservations. L'architecture devra permettre d'ajouter des ressources lorsque la charge augmente.

DISPONIBILITÉ

L'application doit être disponible de manière continue et limiter les interruptions de service. Les composants critiques devront pouvoir être redémarrés ou remplacés sans provoquer une interruption importante du service.

SÉCURITÉ

Les données des clients, des réservations et des paiements doivent être protégées. L'application devra utiliser des communications chiffrées, une gestion sécurisée des mots de passe, une authentification fiable et une gestion sécurisée des secrets.

ACCESSIBILITÉ

L'application devra être conçue selon les principes d'accessibilité numérique afin de permettre son utilisation par les personnes en situation de handicap. L'objectif est de respecter les recommandations du RGAA.

INTERNATIONALISATION

La solution devra pouvoir gérer plusieurs pays, langues, devises et règles métier. Les spécificités propres à chaque marché devront pouvoir être prises en compte sans créer une nouvelle application pour chaque pays.

DONNÉES

Les données devront être centralisées dans un modèle cohérent afin d'éviter les doublons et les différences de structure présentes dans l'existant. La solution devra également permettre la migration progressive des données existantes.

ÉVOLUTIVITÉ ET MAINTENABILITÉ

L'architecture devra être suffisamment modulaire pour permettre l'ajout ou la modification de fonctionnalités sans remettre en cause l'ensemble de l'application. Le choix des technologies devra également faciliter sa maintenance sur le long terme.

API ET COMMUNICATION

Les différents composants de l'application devront communiquer à travers des interfaces standardisées. Les API devront permettre notamment la gestion des utilisateurs, des véhicules, des agences, des réservations et des paiements.

CHAT

Le chat entre le client et le conseiller devra permettre des échanges en temps réel. La solution devra donc prendre en charge une communication bidirectionnelle et conserver l'historique des conversations.

2.2 Architecture cible

L'architecture cible repose sur une approche **microservices** afin de répondre aux limites identifiées dans l'existant, notamment la présence de plusieurs applications monolithiques, la divergence des fonctionnalités et la difficulté à faire évoluer les différentes parties du système indépendamment.

Les fonctionnalités principales sont séparées en plusieurs services : authentification, réservation, paiement et chat. Cette séparation permet à chaque service de **regrouper une responsabilité métier précise** et de pouvoir évoluer indépendamment des autres.

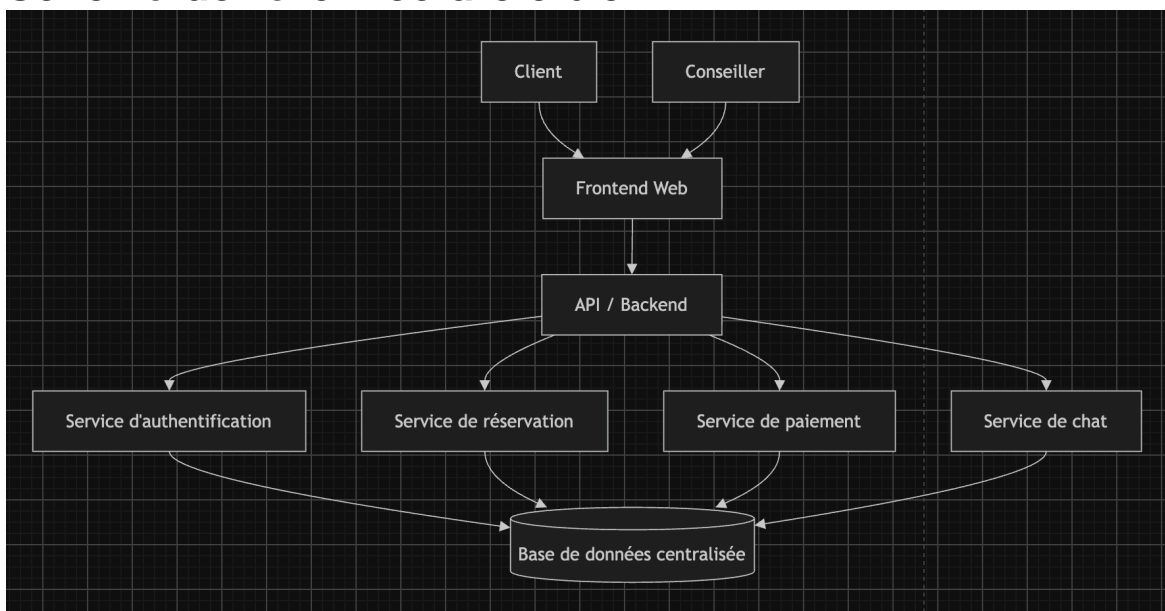
- **Frontend** : interface utilisée par les clients et les conseillers. Il est exécuté dans le navigateur et communique avec le backend via des API. Cette séparation permet de faire évoluer l'interface sans modifier directement la logique métier.
- **Service d'authentification** : gestion des comptes et des accès. Il est séparé afin de centraliser cette responsabilité et de ne pas dupliquer la logique d'authentification dans les autres services.
- **Service de réservation** : gestion de la recherche, création, modification et annulation des réservations. Il regroupe les principales règles métier liées aux réservations afin de pouvoir les faire évoluer indépendamment.
- **Service de paiement** : gestion des paiements. Son isolation permet de limiter les dépendances avec les autres fonctionnalités et de mieux isoler une partie sensible de l'application.
- **Service de chat** : gestion des conversations en temps réel. Il est séparé car le chat possède des contraintes techniques différentes des fonctionnalités classiques, notamment la communication bidirectionnelle en temps réel.

Les différents services communiquent via des API standardisées afin de **limiter leur couplage** et de faciliter leur évolution.

Cette architecture permet ainsi de disposer d'un socle commun pour l'ensemble des pays tout en conservant la possibilité d'adapter certaines règles métier selon les marchés.

Les services pourront être déployés sur plusieurs instances et les requêtes réparties par l'**API Gateway / Load Balancer**, afin d'améliorer la disponibilité et d'absorber les pics de charge.

Schéma de l'architecture cible



Cette architecture permet de disposer d'un socle technique commun pour l'ensemble des pays, tout en conservant la possibilité de gérer leurs différentes règles métier.

2.3 Architecture de déploiement

L'application sera déployée dans une infrastructure cloud pour bénéficier d'une meilleure disponibilité et de pouvoir adapter les ressources en fonction de la charge. Elle sera composée de :

Frontend : application exécutée directement dans le navigateur du client. Cette séparation permet de faire évoluer l'interface indépendamment du backend.

API Gateway / Load Balancer : serveur intermédiaire, par exemple Nginx, qui reçoit les requêtes et les répartit entre les différentes instances du backend. Il améliore la disponibilité et permet de mieux gérer les pics de charge.

Serveurs applicatifs : plusieurs instances exécutant le backend et les API. Elles permettent d'assurer la haute disponibilité et d'augmenter la capacité de l'application en cas de hausse du trafic mais également d'éviter une potentielle panne rendant l'application inutilisable.

Bases de données : chaque service possède sa propre base de données, partagée par les différentes instances de ce service.

Cette organisation suit le principe de responsabilité des données par service : le service de réservation maîtrise ses données de réservation, tandis que le service de paiement maîtrise ses données de paiement.

Cette séparation limite le couplage entre les services et permet de faire évoluer le modèle de données d'un service sans modifier directement celui des autres. Les différentes instances d'un même service utilisent toutefois la même base afin de conserver une donnée cohérente.

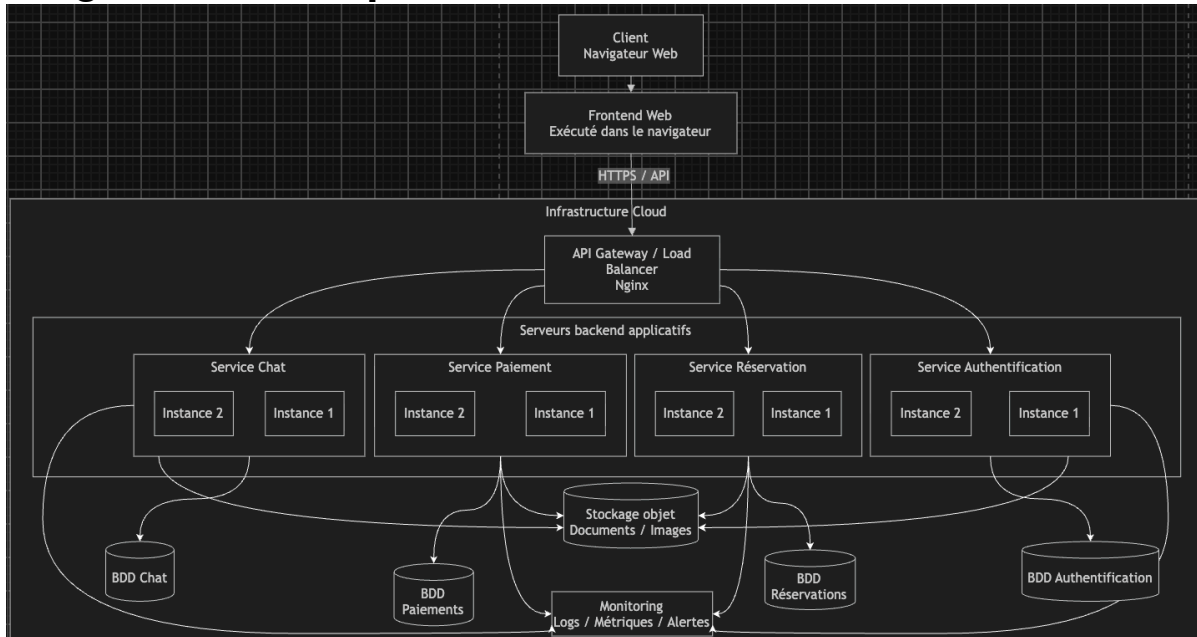
Stockage objet : stockage des fichiers tels que les documents, images ou pièces jointes. Il permet de gérer les fichiers indépendamment de la base de données et de faciliter leur stockage et leur évolution.

Stockage relationnel : les données structurées et liées entre elles sont conservées dans les bases de données relationnelles. Cela permet de garantir la cohérence et l'intégrité des données.

Monitoring : surveillance de l'état et des performances des différents composants. Il permet de détecter rapidement les erreurs, les ralentissements ou les indisponibilités.

HTTPS : sécurisation des communications entre le navigateur et l'application. Il permet de protéger les données échangées contre leur interception.

Diagramme de déploiement



Cette architecture permet d'améliorer la disponibilité, la tolérance aux pannes et la capacité à absorber une augmentation de la charge.

2.4 Modèle de données

Le modèle de données est conçu autour des principales fonctionnalités de l'application : gestion des utilisateurs, des agences, des véhicules, des réservations, des paiements et du chat.

L'objectif est de **centraliser les données communes aux différents marchés** tout en conservant une séparation claire entre les différentes responsabilités métier.

Les principales entités sont :

- **Utilisateur** : contient les informations des clients et des conseillers.
- **Agence** : représente les agences de location et leur localisation.
- **Véhicule** : contient les informations sur les véhicules disponibles à la location.
- **Réservation** : associe un utilisateur à un véhicule et à une agence pour une période donnée.
- **Paiement** : contient les informations liées au paiement d'une réservation.
- **Conversation** : représente une conversation entre un client et un conseiller.
- **Message** : contient les différents messages échangés dans une conversation.

Les relations entre ces entités permettent de conserver une cohérence entre les données. Par exemple, une réservation appartient à un utilisateur et concerne un véhicule, tandis qu'un paiement est associé à une réservation.

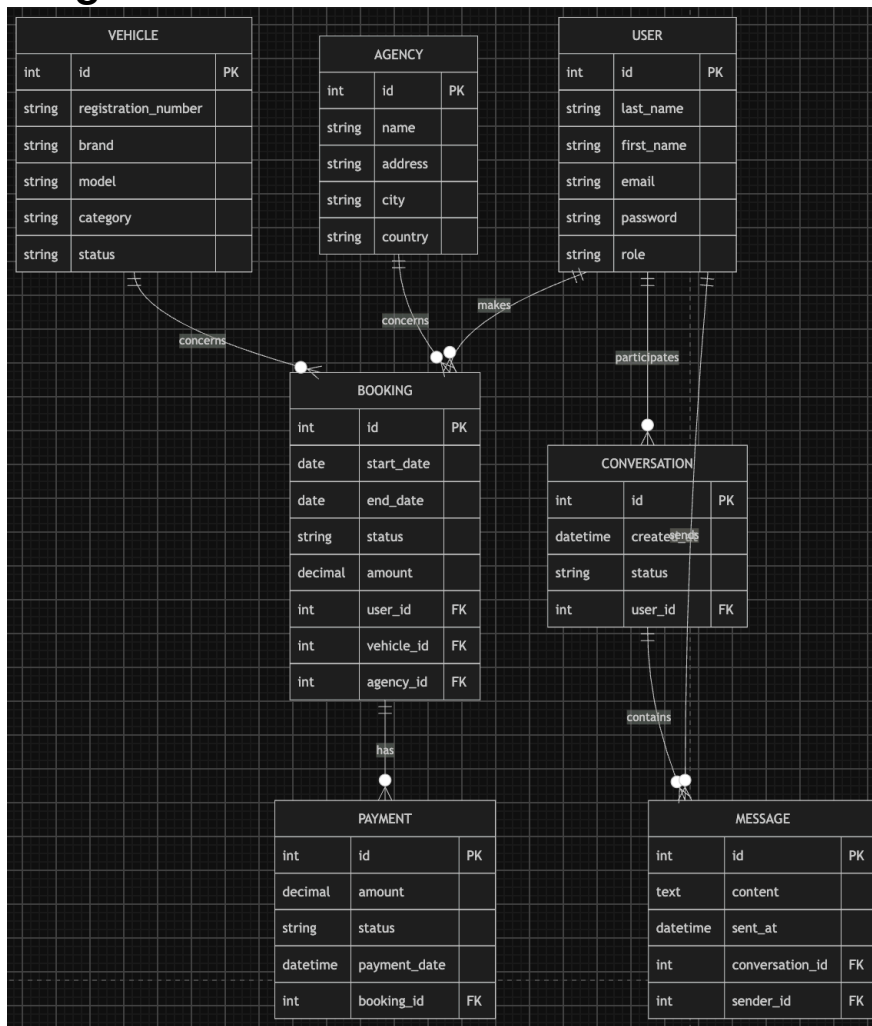
Le modèle de données reprend cette séparation des responsabilités afin de limiter les duplications et de faciliter l'évolution des différents services.

Par exemple, les informations relatives à une réservation sont regroupées dans l'entité **BOOKING**, tandis que les informations concernant l'utilisateur, le véhicule, l'agence et le paiement sont conservées dans leurs entités respectives.

Cette organisation permet de centraliser les données communes à l'ensemble des marchés sans reproduire les modèles de données séparés présents dans l'existant.

Le modèle pourra également être étendu pour intégrer de nouvelles fonctionnalités ou de nouvelles règles propres à certains marchés sans remettre en cause les principales entités.

Diagramme entité-relation



Respect de la troisième forme normale (3NF)

Le modèle de données est conçu selon les principes de la **troisième forme normale (3NF)** afin de limiter la duplication des données et de garantir leur cohérence. Chaque entité représente une information spécifique et ses attributs dépendent directement de sa clé primaire. Les informations qui peuvent être réutilisées sont séparées dans leurs propres entités.

Par exemple, les informations d'une **agence** ne sont pas répétées dans chaque réservation : la réservation référence simplement l'agence grâce à une clé étrangère.

De la même manière, les informations du **véhicule**, de l'**utilisateur** et du **paiement** sont stockées dans leurs entités respectives et reliées par des clés étrangères.

Le modèle évite ainsi les dépendances entre attributs qui ne dépendent pas directement de la clé primaire, ce qui permet de respecter la **3NF** et de faciliter la maintenance et la cohérence des données.

Pourquoi ce modèle ?

Ce modèle permet de **relier les principales données métier sans duplication inutile**. Il facilite notamment la recherche des véhicules disponibles, le suivi des réservations et des paiements, ainsi que l'historique des conversations.

La séparation entre les entités permet également de faire évoluer indépendamment les différentes fonctionnalités de l'application, tout en conservant des relations claires entre les données.

2.5 Choix technologiques

Le choix des technologies repose sur plusieurs critères : **maintenabilité, évolutivité, sécurité, performance et facilité de déploiement**. L'objectif est de retenir des technologies adaptées aux besoins du projet sans ajouter de complexité inutile.

Les choix technologiques sont réalisés en cohérence avec l'architecture cible. Ils doivent permettre de mettre en œuvre la séparation des responsabilités, le déploiement de plusieurs instances et la communication entre les différents services, tout en limitant la complexité technique.

Besoin	Technologie retenue	Justification
Frontend	React	Permet de créer une interface moderne et réutilisable. Sa séparation avec le backend facilite également son évolution.
Backend / API	Laravel	Framework maîtrisé, adapté au développement d'API et disposant de fonctionnalités intégrées pour l'authentification, la validation et la gestion des données.
Base de données	PostgreSQL	Base relationnelle robuste, adaptée aux nombreuses relations entre utilisateurs, véhicules, agences et réservations.
API Gateway / Load Balancer	Nginx	Permet de servir de point d'entrée à l'application et de répartir les requêtes entre plusieurs instances backend.
Authentification	Laravel Sanctum	Solution adaptée à l'authentification d'une application web utilisant une API.
Chat temps réel	WebSockets	Permet une communication bidirectionnelle entre le client et le serveur, nécessaire pour les échanges en temps réel.
Stockage fichiers	Stockage objet	Adapté au stockage des documents, images et pièces jointes sans surcharger la base de données.
Déploiement	Docker	Permet de standardiser l'environnement d'exécution et de faciliter les déploiements entre les différents environnements.
Monitoring	Prometheus + Grafana	Permet de suivre les performances de l'application et de détecter rapidement les problèmes.
Architecture applicative	Microservices	Permet de séparer les responsabilités métier et de faire évoluer ou dimensionner indépendamment les principales fonctionnalités.
Données	Une BDD par service	Limite le couplage entre les services et permet à chaque service de maîtriser son modèle de données.
Scalabilité	Plusieurs instances	Permet d'augmenter la capacité d'un service sans multiplier inutilement les ressources des autres.

Justification des principaux choix

React plutôt qu'Angular

React permet de construire une interface modulaire et réutilisable. Il offre également une séparation claire avec le backend Laravel, ce qui facilite l'évolution de l'interface sans modifier la logique métier.

Laravel plutôt que Node.js ou Java/Spring Boot

Laravel a été privilégié pour sa simplicité de développement, son écosystème complet et son adéquation avec les besoins de l'application. Il permet notamment de gérer facilement les API, l'authentification et l'accès aux données.

PostgreSQL plutôt qu'une base NoSQL

Les données de l'application sont fortement liées entre elles : une réservation concerne un utilisateur, un véhicule et une agence, puis peut être associée à un paiement. Une base relationnelle est donc plus adaptée pour garantir la cohérence et l'intégrité de ces relations.

Docker pour le déploiement

Docker permet d'obtenir des environnements reproductibles et facilite le déploiement de l'application. Il s'intègre également avec l'architecture utilisant plusieurs instances des services.

Nginx comme API Gateway / Load Balancer

Nginx constitue un point d'entrée unique pour les requêtes et permet de les répartir entre les différentes instances. Cela répond directement au besoin d'améliorer la disponibilité et d'absorber les pics de charge.

2.6 Cohérence de l'architecture avec les besoins

L'architecture cible répond aux principaux besoins identifiés lors de l'audit et dans les spécifications techniques.

La **centralisation** permet d'unifier les fonctionnalités et les données des différents marchés. La séparation des services facilite la **maintenabilité et l'évolution** de l'application, tandis que la réplication des services et le **Load Balancer** améliorent sa **disponibilité et sa capacité à absorber les pics de charge**.

Enfin, l'utilisation d'une API commune, d'un modèle de données structuré et de composants dédiés à l'authentification, au paiement et au chat permet de répondre aux besoins fonctionnels tout en conservant une architecture **évolutive et sécurisée**.